



java.util package contains:

- ❖ Legacy classes
- ❖ Collection classes
- ❖ Miscellaneous classes

Don't write or place any
image in this area

If we want to declare 10 int variables:

```
int i1=1, i2=2, i3=3, i4=4, i5=5, i6=6, i7=7, i8=8, i9=9, i10=10;
```

Instead we can array of int type:

```
int i[]=new int[10]; // array declaration
```

```
i[0]=1; // array initialization
```

```
i[1]=2;
```

```
..
```

```
i[9]=10;
```

```
System.out.println(i[7]); // 6 // accessing array element @ index 7
```

Arrays are used to reduce declaring many variables of the same datatype (homogeneous type).

Don't write or place any
image in this area

java.util package classes are introduced as an alternative to arrays. The limitations of array are:

```
int i[]=new int[10];
```

- a) Homogeneous (same datatype)
- b) Fixed in size they cannot grow or shrink, memory wastage
- c) dealing with indexes increases complexity of the program, sometimes leads to error prone for accessing array beyond index

Don't write or place any
image in this area

java.util package



In java.util package we will deal with

- ❖ legacy classes (classes exist from the first version of JDK-JDK 1.0)
- ❖ collection classes (classes introduced in JDK 1.1.x version and include in JDK 1.2 version)
- ❖ and miscellaneous classes (since JDK 1.0, 1.1)

Don't write or place any
image in this area

java.util package collection classes:

- i. they can store objects of different data types, they are "heterogeneous"
- ii. size can grow and shrink, no wastage of memory
- iii. convenient methods are provided to store, retrieve and remove objects from these classes (add(), get(), remove(), contains(), elements())

Don't write or place any
image in this area

java.util.package is introduced in JDK 1.0 (1995) in which limited number of classes are released. They are:

- i. Vector (c) (FIFO)
- ii. Stack (c) (LIFO)
- iii. Dictionary (ac)
 - a. Hashtable (c)
 - i. Properties (c)
- iv. Enumeration (i)
- v. Observable (i)
- vi. EventObject (i)
- vii. Date (c)
- viii. Random (c)
- ix. StringTokenizer (c)

Don't write or place any
image in this area



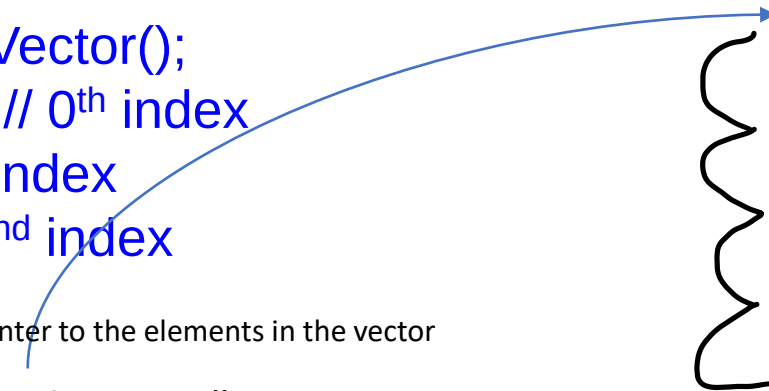
class Vector:

Vector is an incremental integer index based array. The index value starts from 0 ... n-1. In Vector given order is maintained.

```
Vector v1=new Vector();  
v1.add("one"); // 0th index  
v1.add(2); // 1st index  
v1.add(3.0); // 2nd index
```

Enumeration is a pointer to the elements in the vector

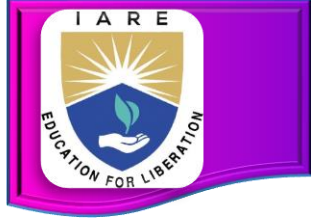
```
Enumeration en=v.elements();  
while(en.hasMoreElements() {  
    Object o=en.nextElement();  
}
```



index	value
0	One
1	2
2	3.0

Don't write or place any
image in this area

java.util package (Legacy Classes)



Vector class methods are:

1. add(Object element)
2. add(int index, Object element)
3. addElement(Object element)
4. Object get(int index)
5. Enumeration elements()
6. int indexOf(Object o)
7. int lastIndexOf(Object o)
8. remove(int index)
9. remove(Object o)
10. set(int index, Object o)
11. Object firstElement()
12. Object lastElement()
13. int size()
14. setSize(int requiredSize)
15. capacity()
16. clear()

17. ensureCapacity(int expectedCapacity)

18. boolean isEmpty()

19. retainAll(Collection c)

20. List subList(int fromIndex, int toIndex-1)

21. Object[] toArray()

Enumeration interface methods are:

1. boolean hasMoreElements()
2. Object nextElement()

Don't write or place any
image in this area

java.util package (Legacy Classes)

// VectorEx1.java

```
import java.util.*;
public class VectorEx1 {
    public static void main(String args[])
        throws Exception {
        Vector v=new Vector();

        v.addElement("First Element");
        v.addElement("Second Element");
        v.addElement("Third Element");
        v.addElement("Fourth Element");

        // v.trimToSize();

        v.addElement("Fifth Element");
        v.addElement("Sixth Element");

        System.out.println(v.get(3));
```

```
Enumeration en=v.elements();
while(en.hasMoreElements()) {
    Object o=en.nextElement();
    System.out.println((String)o);
}

v.add(1,"Second-and-half element ");

en=v.elements();
while(en.hasMoreElements()) {
    Object o=en.nextElement();
    System.out.println((String)o);
}

v.set(1,"");

en=v.elements();
while(en.hasMoreElements()) {
    Object o=en.nextElement();
    System.out.println((String)o);
}
```

```
Object o[]=v.toArray();
int len=o.length;
for(int i=0;i<len;i++) {
    System.out.println((String)o[i]);
}

Collection c=v.subList(0,3);
Iterator i=c.iterator();
while(i.hasNext()) {
    System.out.println(i.next().toString());
}
}
```

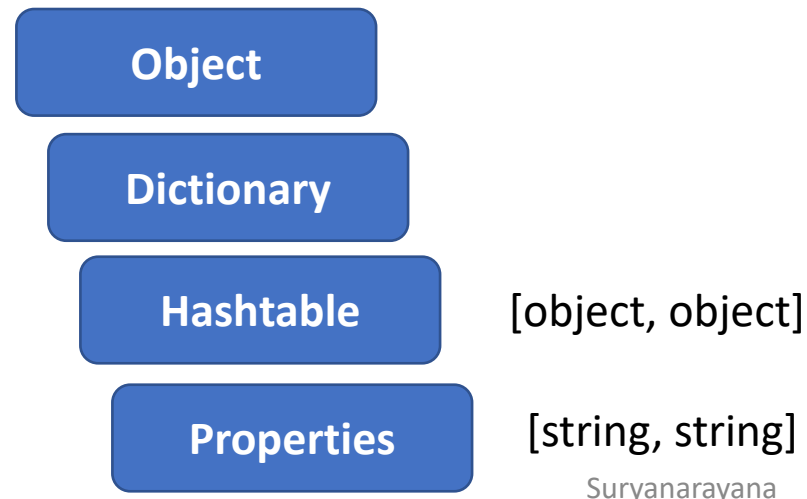
Don't write or place any
image in this area

java.util package (Legacy Classes)



But index based search is always slower because until we go through objects of all the indexes we cannot determine where the desired object exist. Vector paradigm/model is FIFO. Vector allows duplication.

Hence if objects are stored with a easily rememberable key as a index to store the value. Searching object with that user defined key based becomes faster than index based search. The **Dictionary** sub classes are used for user defined key based indexes. One is **Hashtable** and the other one is **Properties** class. In both the objects user defined key hashCode value is stored as index to store values. Hence the given order cannot be guaranteed. Here also the index is an integer, but the integer is not incremental integer index, instead it is given key hashCode based index.



Don't write or place any
image in this area

Vector class

index	value
0	10
1	20
2	30
3	40

There are two types of arrays:

1. index based array (Vector)
2. key based array (Hashtable)

Hashtable & Properties classes

key as index	value
surya	9848111288
abc	9848012345
xyz	9849012345
lmn	6302151056

Don't write or place any
image in this area

java.util package (Legacy Classes)



- ❖ In Hashtable both key and value pairs can be any type of objects.
- ❖ The key value pairs can be stored, retrieved and removed only within the program but cannot be loaded from external properties file
- ❖ Both Hashtable & Properties classes maintains keys as unique and values can be duplicate
- ❖ Properties class also maintains key and value pair but they are called as propertyName and propertyValue
- ❖ Both propertyName and propertyValue are Strings
- ❖ Properties can load properties from an external properties file

Don't write or place any
image in this area

class Hashtable:

Hashtable is a sub class of Dictionary abstract class. Vector is a integer index based array. Hashtable is user defined key's hashCode index based array. Based on the key's hashCode it arranges the objects hence it cannot guarantee the given order. Whichever class object is used as a key, that class is responsible to implement hashCode() method. In Hashtable both key and value can be any type of class objects. Its important methods are:

```
public void put(Object key, Object value)
public Object get(Object key)
public void remove(Object key)
public Enumeration keys() // return keys
public Enumeration elements() // returns values
public Set keySet()
```

```
boolean isEmpty()
void clear()
boolean contains(Object value)
boolean containsKey(Object key)
boolean containsValue(Object value)
int size()
Collection values()
```

Don't write or place any
image in this area

class Properties:

Properties class is derived from Hashtable. In Hashtable both key and values pairs must be stored and retrieved within the program. Whereas in Properties class we can retrieve such key and value pairs from an external .properties extension file. In Properties class both key and value must be String. Its methods are:

```
public void load(InputStream is)
public void load(Reader r)
public void store(OutputStream os, String
comments)
public void store(Writer w, String
comments)
public void list(PrintStream ps)
public void list(PrintWriter pw)
```

```
public void setProperty(String key, String value)
public String getProperty(String key)
public void removeProperty(String key)
Enumeration propertyNames() // returns keys
Set<String> stringPropertyNames() // returns keys
```

Don't write or place any
image in this area



java.util package (Legacy Classes)



interface Enumeration:

Enumeration is an interface. Enumeration is not a data-structure like Vector or Hashtable or Properties class. Instead it only acts as pointer to values in a Vector or keys or propertyNames in a Hashtable or Properties class. Enumeration interface is having two methods called

- ❖ boolean hasMoreElements()
- ❖ Object nextElement()

Don't write or place any
image in this area



java.util package (Legacy Classes)



// HashtableEx2.java

```
import java.util.*;

public class HashtableEx2 {
    public static void main(String args[]) throws Exception {
        Hashtable ht=new Hashtable();
        ht.put("1st down", "Surya");
        ht.put("2nd down", "ABC");
        ht.put("3rd down", "XYZ");
        ht.put("4th down", "LMN");
        ht.put("5th down", "PQR");

        System.out.println(ht.get("3rd down"));

        // returns all the keys as a Enumeration object
        Enumeration en=ht.keys();
        while(en.hasMoreElements()) {
            String key=en.nextElement().toString();
            String value=ht.get(key).toString();
            System.out.println(key+" "+value);
        }
        System.out.println("*****");
    }
}
```

```
// returns all the values as a Enumeration object
en=ht.elements();
while(en.hasMoreElements()) {
    String value=en.nextElement().toString();
    System.out.println(value);
}
System.out.println("*****");

Set set=ht.keySet();
Iterator i=set.iterator();
while(i.hasNext()) {
    String key=i.next().toString();
    String value=ht.get(key).toString();
    System.out.println(key+" "+value);
}
}
```

Don't write or place any
image in this area



java.util package (Legacy Classes)



// PropertiesEx3.java

```
import java.util.*;
import java.io.*;
public class PropertiesEx3 {
    public static void main(String args[]) throws Exception {

        Properties p=new Properties();

        p.load(new FileInputStream("battingOrder.properties"));

        Enumeration e=p.propertyNames();
        while(e.hasMoreElements()) {
            String key=(String)e.nextElement();
            String value=p.getProperty(key);
            System.out.println(key+" "+value);
        }
    }
}
```

Properties class example

Object

|

Dictionary

|

Hashtable

|

Properties

battingOrder.properties

1st_down=Surya

2nd_down=Sakib

3rd_down=Habib

4th_down=Mamun

5th_down=Saud

```
System.out.println("*****");
Set<String> keys=p.stringPropertyNames();
for(String key:keys) {
    String value=p.getProperty(key);
    System.out.println(key+" "+value);
}
System.out.println("*****");

p.setProperty("6th_down","Tahsin");

p.store(new FileOutputStream(
"battingOrder.properties"), "6thdown added");
}
}
Suryanarayana
```

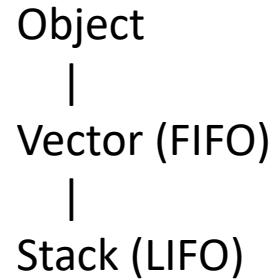
Don't write or place any
image in this area



java.util package (Legacy Classes)



class Stack



Stack super class is Vector/Stack is a sub class of Vector. Stack represents LIFO/FILO order of arranging objects. Stack allows duplicate. Five additional methods are given in this class. They are:

- ❖ Object push(Object o) - adds element on top
- ❖ Object pop() - return and removes top element
- ❖ Object peek() - return top element but doesn't remove
- ❖ int search(Object o) - returns index of the element, returns -1 if element not found
- ❖ boolean empty() - return true is stack is empty

Don't write or place any
image in this area



java.util package (Legacy Classes)



// StackEx4.java

```
import java.util.*;
public class StackEx4 {
    public static void main(String args[]) {
        Stack s=new Stack();
        System.out.println(s.capacity());
        System.out.println(s.size());
        s.push("Surya");
        s.push("Sakib");
        s.push("Habib");
        s.push("Mamun");
        s.push("Saud");
        System.out.println(s);

        System.out.println(s.capacity());
        System.out.println(s.size());

        while(!s.empty()) {
            System.out.println(s.pop());
        }
    }
}
```

```
// alternatively we can write
while(s.empty()!=false) {
}
```

Don't write or place any
image in this area



Collection API

Don't write or place any
image in this area



java.util package (Collection API)



- ❖ Classes given in JDK 1.2 java.util package classes are called as Collection Framework classes.
- ❖ Classes exist in java.util package before JDK 1.2 are called as Legacy/Old classes.

The main motto behind developing Collection framework is in legacy classes (Vector, Hashtable, Properties) the methods are declared as synchronized. Because the legacy class methods are declared as synch, in multithreaded environment developer cannot remove synch of the methods due to which performance impact. In multi-threaded environment the legacy class run like a thread-safe.

Wherever legacy classes are running in a multi-threaded environment where performance is priority more than consistency of object's state there we can use Collection API.

All Collection interfaces and classes - methods are not synchronized by default. If we want the Collection objects to be synchronized we can synch in multi-threaded environment using

- ✓ `Collection Collections.synchronizedCollection(Collection)`
- ✓ `List Collections.synchronizedList(List)`
- ✓ `Set Collections.synchronizedSet(Set)`
- ✓ `Map Collections.synchronizedMap(Map)`

Don't write or place any
image in this area



java.util package (Collection API)



- ❖ Classes given in JDK 1.2 java.util package classes are called as Collection Framework classes.
- ❖ Classes exist in java.util package before JDK 1.2 are called as Legacy/Old classes.

The main motto behind developing Collection framework is in legacy classes (Vector, Hashtable, Properties) the methods are declared as synchronized. Because the legacy class methods are declared as synch, in multithreaded environment developer cannot remove synch of the methods due to which performance impact. In multi-threaded environment the legacy class run like a thread-safe.

Wherever legacy classes are running in a multi-threaded environment where performance is priority more than consistency of object's state there we can use Collection API.

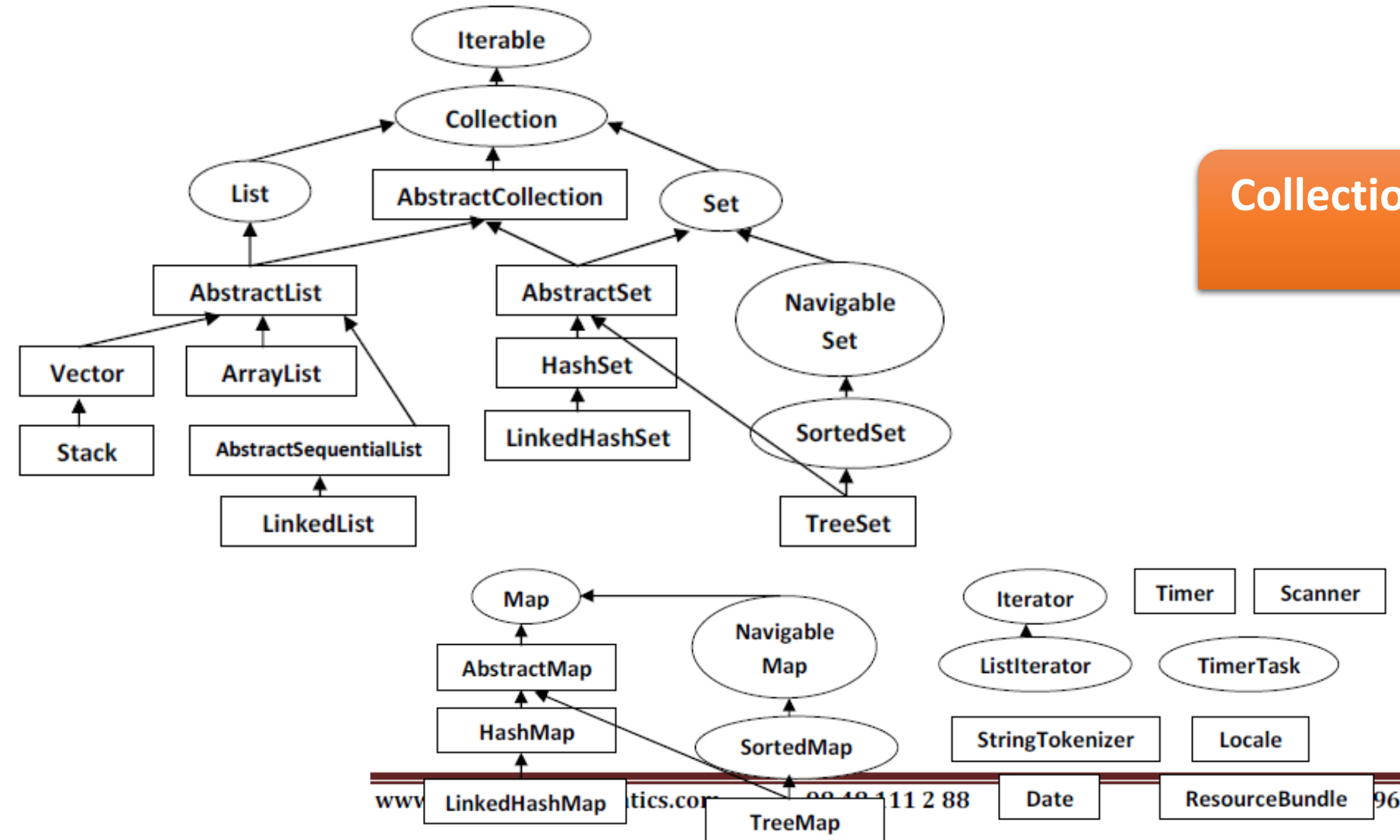
All Collection interfaces and classes - methods are not synchronized by default. If we want the Collection objects to be synchronized we can synch in multi-threaded environment using

- ✓ `Collection Collections.synchronizedCollection(Collection)`
- ✓ `List Collections.synchronizedList(List)`
- ✓ `Set Collections.synchronizedSet(Set)`
- ✓ `Map Collections.synchronizedMap(Map)`

Don't write or place any
image in this area



Collection classes hierarchy diagram



Don't write or place any image in this area



ArrayList: Ordered, Allows duplication, slow

LinkedList: Ordered, Allows duplication, fast

HashSet: Unique, Unordered

LinkedHashSet: Unique, Given order

TreeSet: Unique, Natural order

HashMap: Unique keys, Unordered

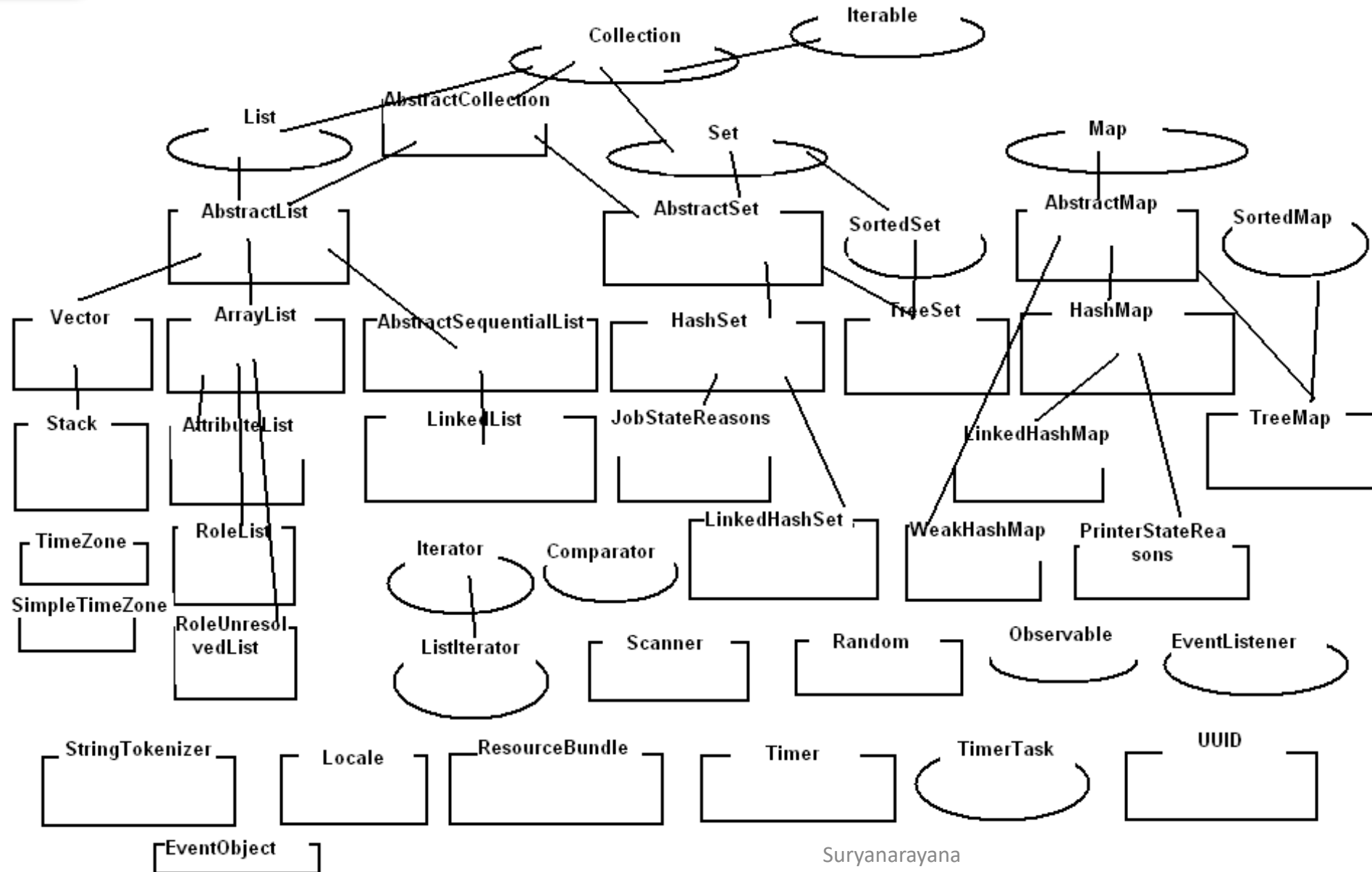
LinkedHashMap: Unique keys, Given order

TreeMap: Unique keys, Natural order

**Collection classes hierarchy
diagram**

Don't write or place any
image in this area

java.util package (Collection API)



Don't write or place any image in this area



java.util package (Collection API)



Legacy classes Vs Collection classes

1. Legacy class methods are synch
Collection class methods are not synch. They can be made synch by calling List
Collections.synchronizedList(List)
2. Legacy class don't allow null as a key or value
Whereas Collection allows null as both key and value
3. Legacy classes serialized form is not cross compatible with other language data structures
Collection classes object serialization are inter-operable with C++ STL (Standard Template Library)
4. Legacy class growth percentage/load-factor remain constant on all JVM
Collection load-factor varies from client to server VM.
5. Client VM can be started (java -client ClassName)
Server VM can be started (java -server ClassName)
6. Whenever we give Legacy class objects to other they are modifiable. Mutator objects.
Collection objects are also mutable by default, but we can make them immutable by passing them into utility methods. Collection objects can be made read-only.
List Collections.unmodifiableList(list)

Don't write or place any
image in this area



java.util package (Collection API)



Collection without Generic (meant for heterogenous objects):

```
Set s=new HashSet();
s.add("Saud");
s.add("Habib");
Integer in=new Integer(10);
s.add(in);

Iterator i=s.iterator();
while(i.hasNext()) {
    Object o=i.next();
    if(o instanceof String) {
        String str=(String)o;
        System.out.println(str);
    } else if(o instanceof Integer){
        Integer in=(Integer)o;
        System.out.println(in.intValue());
    } else {
        System.out.println(o);
    }
}
```

Don't write or place any
image in this area



java.util package (Collection API)



Collection with Generic (mean storing homogenous objects):

```
Set<String> s=new HashSet<String>();  
s.add("Saud");  
s.add("Habib");  
// s.add(new Integer(10)); // compilation error saying Integer is not a String
```

// one way of iterating over collection

```
Iterator<String> i=s.iterator();  
while(i.hasNext()) {  
    String str=i.next(); // typecasting is avoided; auto unboxing  
    System.out.println(str);  
}
```

// another way of iterating over collection

```
for(String str:s) {  
    System.out.println(str);  
}
```

Don't write or place any
image in this area



java.util package (Collection API)



interface Collection

Collection is the super interface to all the collection sub classes. Especially List and Set interface are derived from Collection interface. Map and its sub classes are part of Collection API but not sub classes of Collection interface.

The two immediate sub interfaces of Collection are List and Set

List is a ordered collection. List maintains the objects in given order. List is a incremental integer index based array. List allows duplications and accepts any number of null values.

Set is a unique collection. Set don't maintain given order. It stores object based on their object hashCode as a index for that HashSet uses HashMap to store object hashCode as key and given object as a value. Set prevents duplications and allows only one null value.

SortedSet is unique collection cum maintains natural order. The sub class is TreeSet. Objects which are adding into TreeSet must be implemented from Comparable interface.

Don't write or place any
image in this area



java.util package (Collection API)



Methods of Collection interface are:

add(Object o)
addAll(Collection c)
Iterator iterator()
remove(Object o)
removeAll(Collection c)
retainAll(Collection c)
Object[] toArray()

int size()
clear()
isEmpty()
boolean contains(Object o)
boolean containsAll(Collection c)

Note: Object get(int index), remove(int index) are not given in Collection interface, why?
Because Collection interface methods are common to both List and Set classes, because **Set** classes don't use integer index hence those index based methods are not given Collection interface.

Don't write or place any
image in this area



java.util package (Collection API)



Methods of "List" interface are:

- ❖ add(int atIndex, Object newO)
- ❖ addAll(int atIndex, Collection c)
- ❖ Object get(int index)
- ❖ int indexOf(Object o)
- ❖ int lastIndex(Object o)
- ❖ remove(int index)
- ❖ remove(Object o)
- ❖ set(int index, Object o)
- ❖ List subList(int fromIndex, int toIndex)
- ❖ ListIterator listIterator()

Methods of "Set" interface are:

Set interface methods are same as Collection interface methods.

Don't write or place any
image in this area



Methods of SortedSet interface are:

- ❖ Comparator comparator()
- ❖ Object first()
- ❖ Object last()
- ❖ SortedSet headSet(Object o)
- ❖ SortedSet tailSet(Object o)
- ❖ SortedSet subSet(Object fromElement, Object toElement)

AbstractList:

Provided generic implementation to all the List interface methods except to get() method.

Don't write or place any
image in this area



ArrayList

public class ArrayList extends AbstractList
implements Cloneable, RandomAccess, Serializable

Having 3 constructors

ArrayList()
ArrayList(Collection c)
ArrayList(int initialCapacity)

Additional methods of ArrayList class are:

Object clone()
ensureCapacity(int expectedCapacity)
int size()
trimToSize()

Don't write or place any
image in this area



AbstractSet

Contains one protected no parameter constructor
equals(), hashCode() and removeAll(Collection c) methods are implemented.
No default implementation to Collection and Set interface methods are provided.

HashSet

They are first time implemented in HashSet class. Methods implemented are

add(Object o)
clear()
clone()
contains(Object o)
isEmpty()
iterator()
remove(Object o)
size()

Constructors are:

HashSet()
HashSet(Collection c)
HashSet(int initialCapacity)
HashSet(int initialCapacity, float loadFactor)

Don't write or place any
image in this area



LinkedList

public class LinkedHashSet extends HashSet

No methods are overridden. But its default constructor allocates memory for 16 elements with a default load factor of 0.75

Hashtable with LinkedList implementation version of Set is LinkedHashSet. Every element maintains its previous and next element address, hash functionality doesn't influence here. Given order will be maintained and allows multiple null elements. Not-null objects are unique.

add(), remove() and contains() operations are expensive than HashSet because of reordering object positions.

Don't write or place any
image in this area



java.util package (Collection API)



Collections

Collections is a utility API. It consists of static methods that operates on Collection or return Collection objects. Some important methods are:

```
static <T> int binarySearch(List<? extends Comparable<? super T>> list, T key)
static <E> Collection<E> checkedCollection(Collection<E> c, Class<E> type)
static <E> List<E> checkedList(List<E> c, Class<E> type)
static <E> Set<E> checkedSet(Set<E> s, Class<E> type)
static <K,V> Map<K,V> checkedMap(Map<K,V> m, Class<K> keyType, Class<V> valueType)
static <T> void copy(List<? super T> dest, List<? extends T> src)
static boolean disjoint(Collection<?> c1, Collection<?> c2)
static <T> Enumeration<T> emptyEnumeration()
static <T> Iterator<T> emptyIterator()
static final <T> List<T> emptyList()
static <T> ListIterator<T> emptyListIterator()
static final <K,V> Map<K,V> emptyMap()
static final <T> Set<T> emptySet()
static <T> void fill(List<? super T> list, T obj)
static int frequency(Collection<?> c, Object o)

static int indexOfSubList(List<?> source, List<?> target)
static int lastIndexOfSubList(List<?> source, List<?> target)
static <T> T max(Collection<? extends T> coll, Comparator<? super T> comp)
static <T> T min(Collection<? extends T> coll, Comparator<? super T> comp)
static <T> boolean replaceAll(List<T> list, T oldVal, T newVal)
static void reverse(List<?> list)
static void rotate(List<?> list, int distance)
static void shuffle(List<?> list)
static void shuffle(List<?> list, Random rnd)
static <T extends Comparable<? super T>> void sort(List<T> list)
static void swap(List<?> list, int i, int j)
static <T> Collection<T> synchronizedCollection(Collection<T> c)
static <T> List<T> synchronizedList(List<T> list)
static <K,V> Map<K,V> synchronizedMap(Map<K,V> m)
static <T> Set<T> synchronizedSet(Set<T> s)
static <T> Collection<T> unmodifiableCollection(Collection<? extends T> c)
static <T> List<T> unmodifiableList(List<? extends T> list)
static <K,V> Map<K,V> unmodifiableMap(Map<? extends K,? extends V> m)
static <T> Set<T> unmodifiableSet(Set<? extends T> s)
```

Don't write or place any
image in this area



java.util package (Collection API)



Example on ArrayList

// ALDemo.java

```
import java.util.*;
public class ALDemo {
    public static void main(String args[]) throws Exception {

        ArrayList<String> list=new ArrayList<String>();

        list.add("Apple");
        list.add("Mango");
        list.add("Guava");
        list.add("Pineapple");
        list.add("Grapes");
        list.add("Orange");
        System.out.println(list);

        // String fruit=list.get(0);
        String fruit=list.get(0); // list.get(0) returns object, it must be type casted into
        String explicitly but because during ArrayList declaration we applied String as
        generic we need not have to do the type casting.

        System.out.println(fruit);
        System.out.println("*****");

        // old style of fetching data from array list
        Iterator<String> i=list.iterator();
        while(i.hasNext()) {
            String fru=i.next(); // auto unboxing
            System.out.println(fru);
        }
        System.out.println("*****");

        // fetching data using JDK 1.5 style of enhanced for loop/for-each loop
        for(String fru:list) {
            System.out.println(fru);
        }
        System.out.println("*****");

        Collections.sort(list);
        for(String fru:list) {
            System.out.println(fru);
        }
        System.out.println("*****");
```

```
list.add(4,"Watermelon");
for(String fru:list) {
    System.out.println(fru);
}

System.out.println("*****");

Collections.sort(list);
for(String fru:list) {
    System.out.println(fru);
}

System.out.println("*****");

List<String> list1=Collections.unmodifiableList(list);
// list1.add("Papaya"); // UnsupportedOperationException

List<Integer> list2=new ArrayList<Integer>();
list2.add(new Integer(10));
list2.add(50); // int to Integer object conversion is autoboxing
list2.add(20);
list2.add(40);
list2.add(30);

Iterator<Integer> i2=list2.iterator();
while(i2.hasNext()) {
    int ii=i2.next(); // auto unboxing
    System.out.println(ii);
}

System.out.println("*****");

List<Integer> list3=new ArrayList<Integer>();
list3.add(10);list3.add(20);list3.add(30);list3.add(100);

list2.retainAll(list3);

for(int iii:list2) {
    System.out.println(iii);
}

System.out.println("*****");
```

```
ListIterator<Integer> li=list2.listIterator();
li.add(70); li.add(60); li.add(80);
System.out.println("*****Forward Fetch*****");
while(li.hasNext()) {
    int i1=li.next();
    System.out.println(i1);
}

System.out.println("*****Reverse Fetch*****");
while(li.hasPrevious()) {
    int i1=li.previous();
    System.out.println(i1);
}

System.out.println("*****verifying list2 Fetch*****");
for(int iii:list2) {
    System.out.println(iiii);
}
}
```

Don't write or place any
image in this area



java.util package (Collection API)



// Example on HashSet (Unique and unordered)

// HashSetDemo.java

```
import java.util.*;

public class HashSetDemo {
    public static void main(String args[]) {
        Set<String> set=new HashSet<String>();
        set.add("Apple");
        set.add("Mango");
        set.add("Guava");
        set.add("Pineapple");
        set.add("Grapes");
        set.add("Mango");

        for(String str:set) {
            System.out.println(str+" "+str.hashCode());
        }
    }
}
```

Note: Please refer API documentation on Collections class and ListIterator interfaces

Set maintains uniqueness but no order. Sometimes we want uniqueness with given order or natural order.

- ✓ TreeSet for unique and natural order
- ✓ LinkedHashSet for unique and given order

Don't write or place any
image in this area



java.util package (Collection API)



```
// IntTreeSetDemo.java
import java.util.*;

public class IntTreeSetDemo {
    public static void main(String args[]) {
        TreeSet<Integer> ts=new TreeSet<Integer>();
        // converting primitive values into wrapper class objects
        // is called as autoboxing
        ts.add(new Integer(3));
        ts.add(4); // auto boxing; converting primitive int into
        // Integer type object
        ts.add(2);
        ts.add(5);
        ts.add(1);
    }
}
```

```
Iterator<Integer> i=ts.iterator();
while(i.hasNext()) {
    /*
    Object o=i.next();
    Integer in=(Integer)o; // manual unboxing
    int inn=in.intValue();
    */
    int inn=i.next(); // auto unboxing, converting object
    // into int
    System.out.println(inn);
}

// iterating Collection using enhanced for loop / for-each loop
for(int i:ts) { // auto-unboxing
    System.out.println(i);
}
}
```

Don't write or place any
image in this area



java.util package (Collection API)



Note: While adding primitive values and String we need not have to write any special implementation before adding these objects into TreeSet. But while inserting user defined objects into TreeSet the user defined classes must be implemented from Comparable interface, compareTo() method must be implemented.

```
interface java.lang.Comparable {  
    int compareTo(Object o);  
}
```

Don't write or place any
image in this area



java.util package (Collection API)



// Example on TreeSet

// Emp.java

```
public class Emp implements Serializable, Comparable {
```

```
    int eid;
```

```
    String ename;
```

```
    double sal;
```

```
    String desig;
```

```
    public Emp() {}
```

```
    public Emp(int eid, String ename, double sal, String desig) {
```

```
        this.eid=eid;
```

```
        this.ename=ename;
```

```
        this.sal=sal;
```

```
        this.desig=desig;
```

```
    }
```

```
    public void setEid(int eid) {
```

```
        this.eid=eid;
```

```
    }
```

```
    public int getEid() {
```

```
        return eid;
```

```
    }
```

```
    public void setEname(String ename) {
```

```
        this.ename=ename;
```

```
    }
```

```
    public String getEname() {
```

```
        return ename;
```

```
    }
```

```
    public void setSal(double sal) {
```

```
        this.sal=sal;
```

```
    }
```

```
    public double getSal() {
```

```
        return sal;
```

```
    }
```

```
    public void setDesig(String desig) {
```

```
        this.desig=desig;
```

```
    }
```

```
    public String getDesig() {
```

```
        return desig;
```

```
    }
```

```
    public String toString() {
```

```
        return eid+" "+ename+" "+sal+" "+desig;
```

```
    }
```

```
    public int hashCode() {
```

```
        return toString().hashCode();
```

```
    }
```

Suryanarayana

```
    public int compareTo(Object o) {
```

```
        Emp e=(Emp)o;
```

```
        if(this.eid > e.eid)
```

```
            return +1;
```

```
        else if (this.eid < e.eid)
```

```
            return -1;
```

```
        else
```

```
            return 0;
```

Don't write or place any
image in this area



java.util package (Collection API)



```
// TreeSetDemo.java
import java.util.*;
public class TreeSetDemo {
    public static void main(String args[]) {
        TreeSet<Emp> ts=new TreeSet<Emp>();

        Emp e1=new Emp(3, "ABC", 40000.00, "Designer");
        Emp e2=new Emp(2, "XYZ", 20000.00, "Developer");
        Emp e3=new Emp(1, "LMN", 10000.00, "Sr
Developer");
        Emp e4=new Emp(4, "PQR", 30000.00, "DB
Designer");

        ts.add(e1);
        ts.add(e2);
        ts.add(e3);
        ts.add(e4);
```

```
        Iterator<<Emp> i=ts.iterator();
        while(i.hasNext()) {
            /* Object o=i.next();
            Emp e=(Emp)o; */
            Emp e=i.next();
            System.out.println(e);
        }

        for(Emp ee:ts) {
            System.out.println(ee);
        }
```

Don't write or place any
image in this area



java.util package (Collection API)



Single point about Comparable interface:

Class whose objects want to be natural sorted in TreeSet and TreeMap are derived from Comparable interface.

Sub classes of Comparable interface must implement compareTo() method.

In the compareTo() method any one of the property of user defined class can be used for natural order comparison. Usually eid in Emp, sid in Student, cid in Customer etc.

Don't write or place any
image in this area



java.util package (Collection API)



// EnameComparator.java

```
import java.util.*;

public class EnameComparator implements Comparator {
    Emp ee1;
    Emp ee2;

    public int compare(Object lhs, Object rhs) {
        int i=0;
        if(lhs instanceof Emp && rhs instanceof Emp) {
            ee1=(Emp)lhs;
            ee2=(Emp)rhs;
            i=ee1.ename.compareTo(ee2.ename);
        }
        return i;
    }
    public boolean equals(Object o) {
        return ee1.ename.equals(ee2.ename);
    }
}
```

Don't write or place any
image in this area



java.util package (Collection API)



// SalComparator.java

```
import java.util.*;

public class SalComparator implements Comparator {
    Emp ee1;
    Emp ee2;

    public int compare(Object lhs, Object rhs) {
        int i=0;
        if(lhs instanceof Emp && rhs instanceof Emp) {
            ee1=(Emp)lhs;
            ee2=(Emp)rhs;
            if(ee1.sal > ee2.sal)
                i=1;
            else if(ee1.sal < ee2.sal)
                i=-1;
            else
                i=0;
        }
        return i;
    }

    public boolean equals(Object o) {
        return (ee1.sal == ee2.sal);
    }
}
```

Suryanarayana

Don't write or place any
image in this area



java.util package (Collection API)



// DesigComparator.java

```
import java.util.*;

public class DesigComparator implements Comparator {
    Emp ee1;
    Emp ee2;

    public int compare(Object lhs, Object rhs) {
        int i=0;
        if(lhs instanceof Emp && rhs instanceof Emp) {
            Emp e1=(Emp)lhs;
            Emp e2=(Emp)rhs;
            ee1=e1;
            ee2=e2;
            i=e1.desig.compareTo(e2.desig);
        }
        return i;
    }

    public boolean equals(Object o) {
        return ee1.desig.equals(ee2.desig);
    }
}
```

Don't write or place any
image in this area



java.util package (Collection API)



// EnameComparatorTreeSetDemo.java

```
import java.util.*;
public class EnameComparatorTreeSetDemo {
    public static void main(String rags[]) {
        TreeSet<Emp> ts=new TreeSet<Emp>(new EnameComparator());
        Emp e1=new Emp(3,"ABC",10000.00,"Designer");
        Emp e2=new Emp(2,"XYZ",20000.00,"Developer");
        Emp e3=new Emp(1,"LMN",30000.00,"Sr Developer");
        Emp e4=new Emp(4,"PQR",40000.00,"DB Designer");
        Emp e5=new Emp(2,"XYZ",20000.00,"Developer");
        ts.add(e1); ts.add(e2); ts.add(e3); ts.add(e4); ts.add(e5);
        /*
```

TreeSet will internally call compare method on
EnameComparator object for ename natural sorting.

```
        enc.compare(e2,e1)
        enc.compare(e3,e1)
        enc.compare(e3,e2)
```

```
    */
    for(Emp e:ts) {
        System.out.println(e);
    }
```

```
}
```

Don't write or place any
image in this area



java.util package (Collection API)



// SalComparatorTreeSetDemo.java

```
import java.util.*;

public class SalComparatorTreeSetDemo {
    public static void main(String rags[]) {
        TreeSet<Emp> ts=new TreeSet<Emp>(new SalComparator());
        Emp e1=new Emp(3,"ABC",30000.00,"Designer");
        Emp e2=new Emp(2,"XYZ",20000.00,"Developer");
        Emp e3=new Emp(1,"LMN",40000.00,"Sr Developer");
        Emp e4=new Emp(4,"PQR",10000.00,"DB Designer");
        Emp e5=new Emp(2,"XYZ",50000.00,"Developer");
        ts.add(e1); ts.add(e2); ts.add(e3); ts.add(e4); ts.add(e5);
        /*
            sc.compare(e2,e1);
            sc.compare(e3,e1);
            sc.compare(e3,e2);
        */
        for(Emp e:ts) {
            System.out.println(e);
        }
    }
}
```

Don't write or place any
image in this area



java.util package (Collection API)



```
// DesigComparatorTreeSetDemo.java
import java.util.*;
public class DesigComparatorTreeSetDemo {
    public static void main(String args[]) {
        TreeSet<Emp> ts=new TreeSet<Emp>(new DesigComparator());
        Emp e1=new Emp(3,"ABC",10000.00,"Designer");
        Emp e2=new Emp(2,"XYZ",20000.00,"Developer");
        Emp e3=new Emp(1,"LMN",30000.00,"Sr Developer");
        Emp e4=new Emp(4,"PQR",40000.00,"DB Designer");
        Emp e5=new Emp(2,"XYZ",20000.00,"Developer");
        ts.add(e1); ts.add(e2); ts.add(e3); ts.add(e4); ts.add(e5);
        for(Emp e:ts) {
            System.out.println(e);
        }
    }
}
```

Don't write or place any
image in this area



java.util package (Collection API)



class LinkedList

```
public class LinkedList<E> extends AbstractSequentialList<E>
    implements List<E>, Deque<E>, Cloneable, Serializable
```

LinkedList class derives from AbstractSequentialList (derives from AbstractList) and implements from List, Deque, Cloneable, Serializable interfaces.

Each element in the LinkedList is called the Node. Each Node of the LinkedList contains two items:

- 1) Content of the element
- 2) Pointer/Address/Reference to the Next Node in the LinkedList.

Note: 1. Head of the LinkedList only contains the Address of the First element of the List.

2. The Last element of the LinkedList contains null in the pointer part of the node because it is the end of the List so it doesn't point to anything as shown in the above diagram.

3. The diagram which is shown above represents a singly linked list. There is another complex type variation of LinkedList which is called doubly linked list, node of a doubly linked list contains three parts: 1) Pointer to the previous node of the linked list 2) content of the element 3) pointer to the next node of the linked list.

Don't write or place any
image in this area



java.util package (Collection API)



Why do we need a Linked List?

You must be aware of the arrays which is also a linear data structure but arrays have certain limitations such as:

1. Size of the array is fixed which is decided when we create an array so it is hard to predict the number of elements in advance, if the declared size fall short then we cannot increase the size of an array and if we declare a large size array and do not need to store that many elements then it is a waste of memory.
2. Array elements need contiguous memory locations to store their values.
3. Inserting an element in an array is performance wise expensive as we have to shift several elements to make a space for the new element. For example:

Let's say we have an array that has following elements: 10, 12, 15, 20, 4, 5, 100, now if we want to insert a new element 99 after the element that has value 12 then we have to shift all the elements after 12 to their right to make space for new element.

Similarly deleting an element from the array is also a performance wise expensive operation because all the elements after the deleted element have to be shifted left.

Don't write or place any
image in this area



java.util package (Collection API)



These limitations are handled in the Linked List by providing following features:

1. Linked list allows dynamic memory allocation, which means memory allocation is done at the run time by the compiler and we do not need to mention the size of the list during linked list declaration.
2. Linked list elements don't need contiguous memory locations because elements are linked with each other using the reference part of the node that contains the address of the next node of the list.
3. Insert and delete operations in the Linked list are not performance wise expensive because adding and deleting an element from the linked list doesn't require element shifting, only the pointer of the previous and the next node requires change.

Don't write or place any
image in this area



java.util package (Collection API)



In addition to `AbstractList` methods the additional methods of `LinkedList` are

1. `addFirst(Object o)`
2. `addLast(Object o)`
3. `descendingIterator()`
4. `getFirst()`
5. `getLast()`
6. `offer(Object o) / offerLast(Object)`
7. `offerFirst(Object o)`
8. `Object peek()` retrieves but not remove the head of the element
9. `peekFirst()`
10. `peekLast()`
11. `poll()` FIFO element will be removed- retrieve and removes element from the head of the list
12. `pollFirst()`
13. `pollLast()`
14. `pop()` FILO element will be removed, remove and returns element on the top
15. `removeFirst()`
16. `removeLast()`
17. `removeFirstOccurance(Object o)`
18. `removeLastOccurance(Object o)`

Don't write or place any
image in this area



public interface Queue extends Collection:

A collection designed for holding elements prior to processing. Besides basic Collection operations, queues provide additional insertion, extraction, and inspection operations. Each of these methods exists in two forms: one throws an exception if the operation fails, the other returns a special value (either null or false, depending on the operation). The latter form of the insert operation is designed specifically for use with capacity-restricted Queue implementations; in most implementations, insert operations cannot fail.

Type of Operation	Throws exception	Returns special value
Insert	add(e)	offer(e)
Remove	remove()	poll()
Examine	element()	peek()

Don't write or place any
image in this area



java.util package (Collection API)



The **offer** method inserts an element if possible, otherwise returning false. This differs from the `Collection.add` method, which can fail to add an element only by throwing an unchecked exception. The offer method is designed for use when failure is a normal, rather than exceptional occurrence, for example, in fixed-capacity (or "bounded") queues.

The **remove()** and **poll()** methods remove and return the head of the queue. Exactly which element is removed from the queue is a function of the queue's ordering policy, which differs from implementation to implementation. The `remove()` and `poll()` methods differ only in their behavior when the queue is empty: the `remove()` method throws an exception, while the `poll()` method returns null.

The **element()** and **peek()** methods return, but do not remove, the head of the queue.

Don't write or place any
image in this area



java.util package (Collection API)



public interface Deque<E> extends Queue<E>

A linear collection that supports element insertion and removal at both ends. The name deque is short for "double ended queue" and is usually pronounced "deck". Most Deque implementations place no fixed limits on the number of elements they may contain, but this interface supports capacity-restricted deques as well as those with no fixed size limit.

This interface defines methods to access the elements at both ends of the deque. Methods are provided to insert, remove, and examine the element. Each of these methods exists in two forms: one throws an exception if the operation fails, the other returns a special value (either null or false, depending on the operation). The latter form of the insert operation is designed specifically for use with capacity-restricted Deque implementations; in most implementations, insert operations cannot fail.

Don't write or place any
image in this area



java.util package (Collection API)



The twelve methods described above are summarized in the following table:

The 12 methods for insertion, removal and retrieval of Deque elements are summarized in the following table:

Type of Operation	First Element (Head)		Last Element (Tail)	
	throws e	special value	throws e	special value
Insert	addFirst(e)	offerFirst(e)	addLast(e)	offerLast(e)
remove	removeFirst()	pollFirst()	removeLast()	pollLast()
examine	getFirst()	peekFirst()	getLast()	peekLast()

Don't write or place any
image in this area



java.util package (Collection API)



void addFirst(E e)

Inserts the specified element at the front of this deque if it is possible to do so immediately without violating capacity restrictions, throwing an `IllegalStateException` if no space is currently available. When using a capacity-restricted deque, it is generally preferable to use method `offerFirst(E)`.

boolean offerFirst(E e)

Inserts the specified element at the front of this deque unless it would violate capacity restrictions. When using a capacity-restricted deque, this method is generally preferable to the `addFirst(E)` method, which can fail to insert an element only by throwing an exception.

void addLast(E e)

Inserts the specified element at the end of this deque if it is possible to do so immediately without violating capacity restrictions, throwing an `IllegalStateException` if no space is currently available. When using a capacity-restricted deque, it is generally preferable to use method `offerLast(E)`. This method is equivalent to `add(E)`.

Don't write or place any
image in this area



java.util package (Collection API)



boolean offerLast(E e)

Inserts the specified element at the end of this deque unless it would violate capacity restrictions. When using a capacity-restricted deque, this method is generally preferable to the `addLast(E)` method, which can fail to insert an element only by throwing an exception.

E removeFirst()

Retrieves and removes the first element of this deque. This method differs from `pollFirst` only in that it throws an exception if this deque is empty.

E pollFirst()

Retrieves and removes the first element of this deque, or returns null if this deque is empty.

E removeLast()

Retrieves and removes the last element of this deque. This method differs from `pollLast` only in that it throws an exception if this deque is empty.

E pollLast()

Retrieves and removes the last element of this deque, or returns null if this deque is empty.

Don't write or place any
image in this area



java.util package (Collection API)



E getFirst()

Retrieves, but does not remove, the first element of this deque. This method differs from peekFirst only in that it throws an exception if this deque is empty.

E peekFirst()

Retrieves, but does not remove, the first element of this deque, or returns null if this deque is empty.

E getLast()

Retrieves, but does not remove, the last element of this deque. This method differs from peekLast only in that it throws an exception if this deque is empty.

E peekLast()

Retrieves, but does not remove, the last element of this deque, or returns null if this deque is empty.

Don't write or place any
image in this area



java.util package (Collection API)



Java Linked List example of adding elements:

In the following example we are using add(), addFirst() and addLast() methods to add the elements at the desired locations in the LinkedList, there are several such useful methods in the LinkedList class which I have mentioned at the end of this article.

```
// AddingLinkedListElements.java
```

```
import java.util.*;
```

```
public class AddingLinkedListElements {  
    public static void main(String args[]){
```

```
        LinkedList<String> list=new LinkedList<String>();
```

```
        //Adding elements to the Linked list
```

```
        list.add("Steve");
```

```
        list.add("Carl");
```

```
        list.add("Raj");
```

```
        // Adding an element to the first position  
        list.addFirst("Negan");
```

```
        // Adding an element to the last position  
        list.addLast("Rick");
```

```
        // Adding an element to the 3rd position  
        list.add(2, "Glenn");
```

```
        // Iterating LinkedList  
        Iterator<String> iterator=list.iterator();  
        while(iterator.hasNext()){  
            System.out.println(iterator.next());  
        }  
    }  
}
```

Don't write or place any
image in this area



java.util package (Collection API)



Java example of removing elements from the LinkedList:

In the following example we are checking out the few popular remove methods in the LinkedList that are used to remove elements from certain positions in the LinkedList. Detailed explanation of these methods along with examples are covered in the separate tutorials, links are provided at the end of this article.

```
// RemovingLinkedListElements.java
```

```
import java.util.*;
```

```
public class RemovingLinkedListElements {  
    public static void main(String args[]){  
        LinkedList<String> list=new LinkedList<String>();
```

```
        //Adding elements to the Linked list
```

```
        list.add("Steve");
```

```
        list.add("Carl");
```

```
        list.add("Raj");
```

```
        list.add("Negan");
```

```
        list.add("Rick");
```

```
        // Removing First element
```

```
        // Same as list.remove(0);
```

```
        list.removeFirst();
```

```
        // Removing Last element
```

```
        list.removeLast();
```

```
        // Iterating LinkedList
```

```
        Iterator<String> iterator=list.iterator();
```

```
        while(iterator.hasNext()){
```

```
            System.out.print(iterator.next()+" ");
```

```
        }
```

```
        // removing 2nd element, index starts with 0
```

```
        list.remove(1);
```

```
        System.out.print("\nAfter removing second element: ");
```

```
        // Iterating LinkedList again
```

```
        Iterator<String> iterator2=list.iterator();
```

```
        while(iterator2.hasNext()){
```

```
            System.out.print(iterator2.next()+" ");
```

```
        } } }
```

Suryanarayana

Don't write or place any
image in this area